

DESARROLLO WEB · NODE.JS

Clases ES6, ORM y Sequelize

CRUD de Personas con PostgreSQL

Módulo práctico · **Express + Sequelize + PostgreSQL**
De los objetos de JavaScript a las tablas de la base de datos.

Contenidos

Repaso clases ES6

1. Clases
2. Constructor
3. Métodos e instancia de una clase

Mapeo Objeto-Relacional (ORM)

1. Qué es un ORM
2. Por qué usarlo
3. Ventajas y desventajas
4. Qué es un modelo
5. Qué son las relaciones

Sequelize ORM

1. Qué es Sequelize
2. Instalación y configuración
3. Modelo, atributos y tipos de datos
4. Operaciones CRUD

Proyecto práctico

- CRUD de Personas paso a paso
- API REST con Express
- Conexión real a PostgreSQL

Una **clase** es una plantilla para crear objetos. Define qué **datos** (atributos) y qué **acciones** (métodos) tendrán.

```
class Persona {  
  // se ejecuta al crear el objeto  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
}
```

- Sintaxis moderna y limpia para Programación Orientada a Objetos
- Reemplaza el viejo patrón de funciones constructoras

Constructor, métodos e instancia

01

Definir la clase

```
class Persona {  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
  // método  
  saludar() {  
    return `Hola, soy  
    ${this.nombre}`;  
  }  
}
```

Crear una instancia

```
const p = new Persona("Ana", 30);  
  
p.nombre;      // "Ana"  
p.saludar();   // "Hola, soy Ana"
```

- **constructor:** inicializa los datos
- **método:** función dentro de la clase
- **instancia:** objeto creado con new

¿Qué es un ORM?

02

ORM = Object-Relational Mapping. Es un puente que conecta tus **clases/objetos** con las **tablas** de la base de datos.

- Una **clase** → una **tabla**
- Un **objeto** (instancia) → una **fila**
- Un **atributo** → una **columna**

```
// En vez de escribir SQL a mano:  
// SELECT * FROM personas WHERE id = 1;  
  
// El ORM te deja escribir JavaScript:  
const persona = await Persona.findByPk(1);
```

Por qué usarlo · ventajas y desventajas

02

Ventajas ✓

- Escribes menos SQL repetitivo
- Mismo código sirve para varias bases de datos
- Más seguro contra inyección SQL
- Código más legible y mantenible

Desventajas ✗

- Curva de aprendizaje inicial
- Consultas muy complejas pueden ser lentas
- Abstrae el SQL: a veces no ves qué pasa por debajo

En resumen: el ORM te ahorra tiempo en el 90% de los casos comunes (CRUD).

Un **modelo** es la representación de una tabla dentro de tu código: define el nombre, sus columnas y sus tipos.

Las relaciones conectan modelos entre sí

- **Uno a muchos (1:N):** una Persona tiene muchos Teléfonos
- **Uno a uno (1:1):** una Persona tiene un Pasaporte
- **Muchos a muchos (N:M):** Personas inscritas en muchos Cursos

Hoy: trabajaremos con un solo modelo —**Persona**— para enfocarnos en el CRUD.

¿Qué es Sequelize?

03

Sequelize es el ORM más popular para Node.js. Funciona con PostgreSQL, MySQL, SQLite y más.

- Basado en **promesas** → se usa con `async / await`
- Defines **modelos** y Sequelize genera el SQL por ti
- Incluye validaciones, migraciones y relaciones

// Flujo general

```
1. Conectar → new Sequelize(...)
2. Definir → sequelize.define("Persona", {...})
3. Sincronizar → await sequelize.sync()
4. Usar → Persona.create(), findAll()...
```


1. Instalar paquetes

```
# ORM + driver de PostgreSQL  
npm install sequelize pg pg-hstore  
npm install express dotenv
```

2. Variables (.env)

```
DB_NAME=personas_db  
DB_USER=postgres  
DB_PASS=secret  
DB_HOST=localhost
```

3. Conexión

```
// db.js  
const { Sequelize } =  
require("sequelize");  
  
const sequelize = new Sequelize(  
  process.env.DB_NAME,  
  process.env.DB_USER,  
  process.env.DB_PASS,  
  { host: process.env.DB_HOST,  
    dialect: "postgres" }  
);  
  
module.exports = sequelize;
```

```
// models/Persona.js
const { DataTypes } =
require("sequelize");
const db = require("../db");

const Persona = db.define("Persona",
{
  nombre: {
    type: DataTypes.STRING,
    allowNull: false
  },
  edad: { type: DataTypes.INTEGER },
  email: {
    type: DataTypes.STRING,
    unique: true
  }
});
```

Tipos más usados

- **STRING** → texto corto (VARCHAR)
- **TEXT** → texto largo
- **INTEGER** → números enteros
- **BOOLEAN** → true / false
- **DATE** → fecha y hora

id, **createdAt** y **updatedAt** los crea Sequelize automáticamente.

Operaciones CRUD · Create & Read

03

C — Crear

```
const p = await Persona.create({  
  nombre: "Ana",  
  edad: 30,  
  email: "ana@mail.com"  
});
```

R — Leer

```
// todas  
await Persona.findAll();  
  
// una por id  
await Persona.findByIdPk(1);
```

Recuerda: todas las operaciones devuelven promesas → siempre `await`.

Operaciones CRUD · Update & Delete

03

U — Actualizar

```
const p = await Persona.findByPk(1);
p.edad = 31;
await p.save();

// o directo:
await Persona.update(
  { edad: 31 },
  { where: { id: 1 } }
);
```

D — Eliminar

```
const p = await Persona.findByPk(1);
await p.destroy();

// o directo:
await Persona.destroy({
  where: { id: 1 }
});
```

Proyecto: API REST de Personas

04

Conectamos cada operación CRUD a una **ruta de Express**.

```
const router = express.Router();

router.post("/personas", crear);    // CREATE
router.get("/personas", listar);    // READ all
router.get("/personas/:id", obtener); // READ one
router.put("/personas/:id", editar); // UPDATE
router.delete("/personas/:id", eliminar); // DELETE
```

Patrón REST: mismo recurso (/personas), distinto verbo HTTP según la acción.

Estructura del proyecto

04

```
crud-personas/  
├── .env  
├── package.json  
├── db.js           # conexión  
├── models/  
│   └── Persona.js # modelo  
├── controllers/  
│   └── personas.js # lógica CRUD  
├── routes/  
│   └── personas.js # rutas  
└── index.js       # servidor
```

Cómo ejecutarlo

```
# 1. instalar  
npm install  
  
# 2. crear la BD en postgres  
createdb personas_db  
  
# 3. arrancar  
npm run dev
```

- Servidor en **http://localhost:3000**
- Prueba con **Postman** o **Thunder Client**

RESUMEN

De la clase ES6 a la tabla SQL

Clase → Modelo → Tabla · Objeto → Fila

Lo que logramos hoy: definir un modelo, conectarlo a PostgreSQL y construir un CRUD completo de Personas con Express + Sequelize.

¿Preguntas? Ahora practicamos con el proyecto.